

PESQUISA DE TRABALHOS RELACIONADOS

Extração de features de tráfego de rede em hardware para IDS baseados em aprendizado de máquina

Tema: Módulo em Verilog para Ethernet, IPv4, TCP e UDP

Finalidade: Subsídio à discussão do grupo e à seção de trabalhos relacionados

Método: Revisão direcionada de sete fontes primárias

Conclusão central: o diferencial não é simplesmente usar FPGA em segurança de redes. A contribuição defensável é integrar parsing RTL em Verilog, vetor de features voltado a ML, estímulos de PCAP, oráculo de software e métricas de síntese em uma cadeia reprodutível.

1. Síntese dos trabalhos

Os sete trabalhos a seguir foram escolhidos para ocupar posições diferentes da cadeia de processamento.

1.1 Software: formação de features e datasets

Kitsune (Mirsky et al., 2018) [1]. O sistema realiza detecção online com autoencoders. O extrator mantém 23 estatísticas em cinco janelas com decaimento, totalizando 115 valores, associados a MAC/IP de origem, canal IP e sockets TCP/UDP. Entre as medidas estão contagem, tamanho de pacote, média, dispersão, correlação e jitter. A atualização incremental em tempo constante é uma boa referência para uma futura versão com estado por fluxo. A lacuna é que captura e parsing ocorrem em software, sem RTL sintetizável.

UNSW-NB15 (Moustafa e Slay, 2015) [7]. O dataset foi formado a partir de cerca de 100 GB de PCAP. Argus, Bro/Zeek e algoritmos em C# foram combinados para produzir 49 features e rótulos de nove famílias de ataque. A maior utilidade para o TCC é metodológica: mostra uma cadeia PCAP → ferramentas de referência → tabela de atributos. Esse mesmo princípio pode gerar o ground truth do testbench. Não há implementação em hardware.

1.2 Planos de dados programáveis

P4DDLe (Doriguzzi-Corin et al., 2024) [2]. O plano de dados P4 extrai nove campos por pacote: timestamp, comprimento, flags IP, comprimento TCP, ACK TCP, flags TCP, janela TCP, comprimento UDP e tipo ICMP. Até quatro pacotes são armazenados por fluxo, em janela de dois segundos, e a representação alimenta a CNN LUCID. Filtros de Bloom contadores e bancos duplos de registradores tratam memória e concorrência. É o trabalho funcionalmente mais próximo, mas usa P4/BMv2 e plano de controle, não FPGA/Verilog.

NetBeacon (Zhou et al., 2023) [3]. Implementado em switch Tofino, leva árvores de decisão, Random Forest e XGBoost ao plano de dados. Combina features por pacote - tamanho, TTL e protocolo - com estatísticas por fluxo, como mínimo, máximo, média e variância do tamanho e do intervalo entre pacotes. O artigo evidencia que o comportamento de fluxo pode aumentar a acurácia em tarefas específicas, mas introduz custo de memória e controle de fluxos concorrentes. A plataforma é um ASIC programável, não FPGA RTL.

SketchFeature (Kim et al., 2025) [4]. O sistema extrai, em Tofino-1, distribuições de tamanho de pacote e atraso entre pacotes com estatísticas de terceira ordem. Usa sketch virtualizado e Bloom filter para evitar uma tabela completa por fluxo. Uma configuração reportada ocupa sete dos doze estágios do pipeline e aproximadamente 3 MB de memória. É referência para evolução com features temporais avançadas, mas está além do escopo inicial de parsing determinístico de cabeçalhos.

1.3 FPGA: parsing e detecção

Cabal et al. (2018) [5]. O artigo gera automaticamente um parser VHDL em pipeline a partir de P4, cobrindo Ethernet, VLAN, MPLS, IPv4/IPv6 e TCP/UDP. A arquitetura processa múltiplos pacotes por ciclo e alcança até 1,37 Tb/s em UltraScale+. É a referência mais próxima no aspecto físico e arquitetural. A lacuna é que o parser é genérico: não define vetor de features para IDS/ML, não usa Verilog e não compara a saída com um oráculo derivado de PCAP.

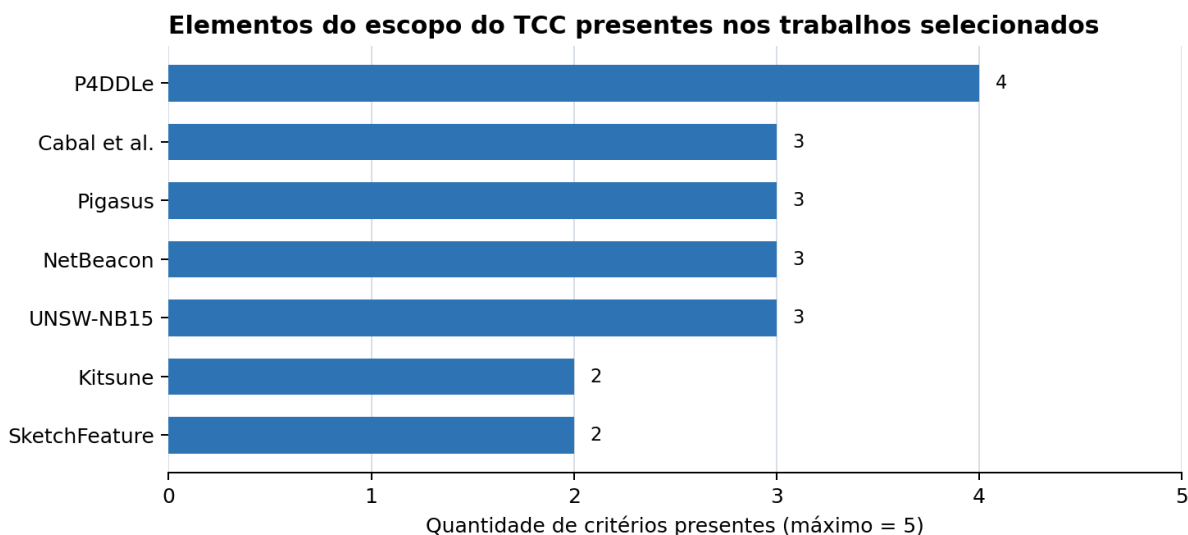
Pigasus (Zhao et al., 2020) [6]. O IDS/IPS FPGA-first executa reassembly, parsing, busca multi-string e parte do casamento de regras em SmartNIC, encaminhando casos suspeitos à CPU. A avaliação reporta 100 Gb/s com, em média, cinco núcleos de CPU e um FPGA. O trabalho comprova a viabilidade de segurança de rede em FPGA, porém utiliza assinaturas e inspeção de payload, e não a formação de vetores para ML.

2. Comparação consolidada

A matriz abaixo resume o componente reaproveitável de cada trabalho e a lacuna que permanece em relação ao TCC.

Trabalho	Plataforma	Proximidade com o TCC	Lacuna principal
P4DDLe	P4 / BMv2	Vetor por pacote para CNN e replay de PCAP	Não é FPGA/Verilog; usa estado e plano de controle
Cabal et al.	FPGA / VHDL	Parser L2-L4 e métricas de síntese/vazão	Parsing genérico; sem vetor ML e sem oráculo
Pigasus	FPGA + CPU	Tráfego real e processamento FPGA a 100 Gb/s	IDS por assinaturas/payload, não features para ML
NetBeacon	Tofino / P4	Features online por pacote e por fluxo	ASIC programável; inclui inferência e estado
UNSW-NB15	Software / dataset	Modelo PCAP → features → rótulos	Pipeline offline, sem hardware
Kitsune	Software online	Front-end incremental para ML	Parser externo; sem RTL/síntese
SketchFeature	Tofino-1 / P4	Features temporais avançadas por fluxo	Complexidade muito além do escopo inicial

Quadro 1 - Comparação dos trabalhos. Fonte: elaboração própria a partir das referências [1]-[7].



Critérios: pacote bruto; parsing L2-L4; FPGA/RTL; saída para IDS/ML; validação PCAP-ground truth.

Figura 1 - Presença de cinco elementos do escopo. A contagem posiciona os trabalhos; não mede qualidade ou desempenho.

3. Diferencial tecnicamente defensável

Formulação recomendada: projetar, implementar e validar um módulo RTL em Verilog que receba um fluxo de bytes de quadros Ethernet, interprete cabeçalhos IPv4/TCP/UDP e emita, com formato e latência determinísticos, um vetor de features para pipelines de IDS baseados em ML. A validação utilizará PCAP e valores de referência obtidos em software; a síntese quantificará recursos, frequência e vazão.

Essa contribuição é defendida pela integração de quatro elementos que normalmente aparecem separados:

- Semântica de protocolos e parsing de bytes em RTL, com tratamento explícito de offsets, endianness e validade.
- Vetor de saída versionado e documentado para consumo por etapas posteriores de ML.
- Ambiente reproduzível PCAP → estímulos → simulação → comparação automática com ground truth de software.
- Caracterização física por síntese: LUTs, FFs, BRAMs, frequência, latência e vazão.

Não se recomenda afirmar que o protótipo é um firewall ou gateway. Sem classificador e sem lógica de ação, ele é corretamente descrito como front-end ou módulo de pré-processamento para IDS.

4. Implicações para o escopo técnico

4.1 Separar features por pacote e por fluxo

Features por pacote podem ser obtidas enquanto o cabeçalho atravessa o módulo. Features por fluxo exigem identificar a 5-tupla, armazenar estado, atualizar estatísticas, expirar entradas e tratar colisões. Essa diferença muda substancialmente a arquitetura.

Recomendação de escopo: entregar primeiro um extrator por pacote e projetar a agregação por fluxo como módulo opcional de segunda etapa.

5.2 Vetor mínimo recomendado

- Ethernet: EtherType; opcionalmente endereços MAC como metadados.
- IPv4: versão, IHL, comprimento total, TTL, protocolo, flags, offset de fragmento e IPs de origem/destino.
- TCP: portas, data offset, flags, número ACK e tamanho da janela.
- UDP: portas e comprimento do datagrama.
- Derivadas por pacote: tamanhos dos cabeçalhos, tamanho estimado do payload, is_tcp/is_udp, fragmentado, inválido e truncado.

IPs e portas podem ser emitidos como metadados, mas convém permitir mascaramento ou exclusão na preparação do dataset para reduzir overfitting à rede de laboratório.

4.3 Interface e condições de erro

- Entrada streaming com sinais data, valid, start-of-packet, end-of-packet e ready.
- Saída com vetor versionado, máscara de validade por campo, features_valid e código de erro.
- Documento de bitfields com offsets, largura, sinal, unidade e endianness de cada feature.
- Comportamento especificado para VLAN, opções IPv4/TCP, fragmentos, protocolo desconhecido, frames truncados e backpressure.

5. Plano de validação e síntese

1. Selecionar ou produzir um PCAP pequeno e controlado, com casos Ethernet/IPv4/TCP e Ethernet/IPv4/UDP.
2. Criar um extrator-oráculo em Python/Scapy ou tshark para gerar, por pacote, o vetor esperado e os indicadores de validade.
3. Converter os bytes do PCAP para o formato de estímulo do testbench, preservando limites de frame e intervalos quando forem relevantes.
4. Executar comparação automática e informar pacote, campo, valor esperado e valor observado em cada divergência.
5. Cobrir casos adversos: IHL variável, opções TCP, UDP sem payload, fragmentação, protocolo desconhecido, truncamento e backpressure.

6. Sintetizar para um FPGA e ferramenta definidos, registrando LUTs, FFs, BRAMs, frequência máxima, ciclos por pacote, latência e vazão teórica.

6. Limitações da revisão

Esta é uma revisão direcionada, não uma revisão sistemática exaustiva. Os trabalhos usam plataformas, datasets e métricas diferentes; portanto, seus valores de acurácia e throughput não devem ser comparados diretamente. A matriz de cinco critérios é uma codificação interpretativa e conservadora para localizar a proposta.

A principal limitação do extrator inicial é a ausência de contexto temporal. Ataques distribuídos, varreduras lentas e comportamentos persistentes costumam exigir features de fluxo ou janelas. O módulo por pacote continua válido como front-end determinístico, mas não deve ser descrito como suficiente para detectar sozinho todas as classes de ameaça.

7. Questões para a reunião

- O produto mínimo será estritamente por pacote ou já deverá manter estado por 5-tupla?
- Qual será a largura do barramento de entrada e qual clock/throughput alvo?
- VLAN, opções IPv4, opções TCP e fragmentação serão suportados, sinalizados ou descartados?
- IPs e portas serão features finais ou metadados opcionais?
- Qual ferramenta produzirá o ground truth: Scapy, tshark, Zeek ou uma combinação?
- Qual FPGA e qual ferramenta de síntese tornarão os resultados reproduzíveis?

Referências

- [1] MIRSKY, Y. et al. Kitsune: An Ensemble of Autoencoders for Online Network Intrusion Detection. NDSS, 2018. DOI 10.14722/ndss.2018.23204. [Acesso online](#)
- [2] DORIGUZZI-CORIN, R. et al. Introducing packet-level analysis in programmable data planes to advance Network Intrusion Detection. Computer Networks, v. 239, 110162, 2024. [Acesso online](#)
- [3] ZHOU, G. et al. An Efficient Design of Intelligent Network Data Plane. 32nd USENIX Security Symposium, 2023, p. 6203-6220. [Acesso online](#)
- [4] KIM, J. et al. SketchFeature: High-Quality Per-Flow Feature Extractor Towards Security-Aware Data Plane. NDSS, 2025. DOI 10.14722/ndss.2025.241071. [Acesso online](#)
- [5] CABAL, J. et al. Configurable FPGA Packet Parser for Terabit Networks with Guaranteed Wire-Speed Throughput. FPGA '18, p. 249-258. [Acesso online](#)
- [6] ZHAO, Z. et al. Achieving 100Gbps Intrusion Prevention on a Single Server. 14th USENIX OSDI, 2020, p. 1083-1100. [Acesso online](#)
- [7] MOUSTAFA, N.; SLAY, J. UNSW-NB15: a comprehensive data set for network intrusion detection systems. MilCIS, 2015. [Acesso online](#)